Week 1 Level 3: Advanced

Save Points: Git Basics

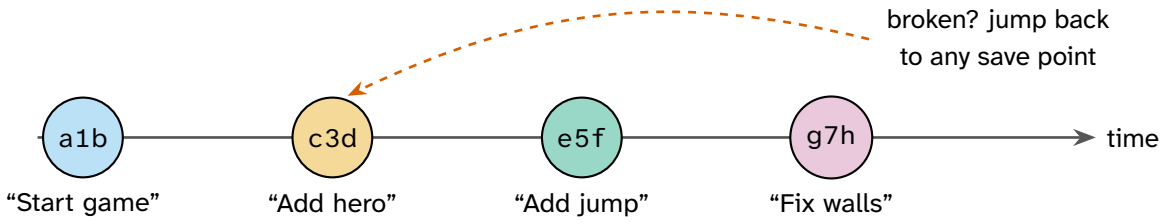
Git is a save-game system for code: every commit is a save point you can go back to.

What you will learn

- ☐ Use Terminal to move between folders.
- ☐ Make save points (commits) with git and give them helpful messages.
- ☐ Look back in time with `git log`, spot changes with `git diff`, and undo mistakes with `git restore`.
- ☐ Warm up your Level 2 skills: functions, lists and loops.

The big idea

In a video game, a save point lets you try something risky. If it goes wrong, you load the save and try again. **Git** gives your code save points. Each save point is called a **commit**. Every commit has a short id (like `a1b2c3d`) and a **message** that says what changed. Together the commits make a timeline: the story of your project.



Saving takes **two moves**, like taking a class photo:



Terminal survival kit

Terminal is a chat window with the computer: type a command, press Return, read the answer.

Type this	It means
pwd	Where am I? (print working directory)
`ls`	List the files in this folder.
`cd git-quest`	Go into the folder `git-quest`.
`cd ..`	Go up one folder.
Tab	Finish a long name for you. Type `cd git-q` and press Tab.
↑	Bring back the last command, so you do not retype it.

The six git commands

Command	What it does	Game words
`git init`	start a save system in this folder	new save file
`git status`	what changed since the last save?	pause menu
`git add hero_badge.py`	choose what goes in the next save	pack your backpack
`git commit -m "Add star"`	make a save point with a message	save the game
`git log --oneline`	list the save points, newest first	load-game menu
`git diff`	show exactly what changed	spot the difference
`git restore hero_badge.py`	throw away unsaved changes	reload last save

Tip: Stuck in a strange screen?

If you typed `git commit` without `-m`, git opens an editor called Vim. Press Esc, type `:q!` and press Return. Then try again with `-m "your message"`.

Good commit messages

A good message finishes the sentence "This commit will …". Start with a verb, keep it short (50 letters or fewer) and say *what* changed.

✗ Not helpful	✓ Helpful
`stuff`	`Add jump to hero`
`update`	`Make star twice as big`
`fixed it`	`Fix score going below zero`

Exercise 1.1: The Git Quest

Folder: `week01/ex-git-quest`. Follow the 7 levels in `README.md`: make a folder called `git-quest` in Documents, set your nickname, `git init`, make your first commit, change the badge color and look at `git diff`, break the file on purpose and rescue it with `git restore`, then make a commit of your own.

Done looks like: `git log --oneline` shows 3 or more commits with helpful messages.

Write your commit ids and messages here (newest first):

Exercise 1.2: Commit Coach

Folder: `week01/ex-commit-coach`. Finish the functions in `starter.py`:

- `is_good_message(message)` returns `False` for empty, too long, boring or one-word messages.
- `make_message("add", "star shape")` returns `"Add star shape"`.
- `find_secrets(lines)` returns the lines that contain words like `password` or `api_key`.

Check with `check_my_work.py`. Then Run it and test 3 messages of your own.

Exercise 1.3: Log Reader (if you finish early)

Folder: `week01/ex-log-reader`. Teach Python to read the text that `git log --oneline` prints. Each line is a commit id, a space, then the message:

```
line = "5b7a913 Fix hero walking through walls"
parts = line.split(" ", 1)   # split only at the FIRST space
```

```
print(parts)   # ['5b7a913', 'Fix hero walking through walls']
```

Finish `parse_log`, `count_commits`, `find_commits` and `oldest_first`. Extra: paste your own `git log --oneline` into SAMPLE_LOG.

Tip: Safety: history remembers everything

Git writes your `user.name` and `user.email` into every commit, forever. Use a nickname and an `@example.com` email. Never commit passwords, real names, addresses or photos of people. Deleting a file later does not delete it from the history.

Vocabulary box

git A program that keeps save points of a folder, so you can look back and undo.

repository (repo) A folder that git is tracking. Git keeps its notes in a hidden `.git` folder inside it.

commit A save point: a snapshot of your files plus a message and an id.

commit message A short label that says what changed, like “Add jump to hero”.

commit id The short code name of a commit, like `a1b2c3d`.

diff The differences between two versions: `-` lines were removed, + lines were added.

Terminal An app where you type commands for the computer.

Reflection

Why does git need two steps (`add`, then `commit`) instead of one?

Write the best commit message you made today, and why it is helpful:

Week 1 Level 3: Advanced

Save Points

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Inventory Review + one commit (about 10 minutes)

1. Open `week01/hw-inventory-review/starter.py` in Thonny. A game backpack is a list like `["gem", "key", "gem"]`.
2. Finish the three TODOs: `count_item`, `remove_all` (make a **new** list) and `most_common`.
3. Open `check_my_work.py` and press Run until every check passes.
4. Git practice: copy your finished `starter.py` into `Documents/git-quest`, then in Terminal:

>_ Terminal

```
cd ~/Documents/git-quest
git add starter.py
git commit -m "Add inventory review"
git log --oneline
```

How many commits does your `git-quest` have now? _____

★★ Challenge (want more?): Message makeover

Rewrite each unhelpful message so that a friend would understand it. Start with a verb.

Unhelpful	Your better message
`stuff` (made the hero jump higher)	
`asdf` (added a sound when you score)	
`fix` (score no longer goes below 0)	

Then use your Commit Coach from class to test your three messages. Do they all say “Great message!”?

Creative (make it yours): A badge with a history

Change `hero_badge.py` in your `git-quest` folder at least 3 times: new colors from the palette, a bigger star, a second ring, your own hero name (made up, never your real name). Commit after **every** change. Then draw your timeline below: one circle per commit with its message.

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-inventory-review`.
- ☐ My `git log --oneline` shows at least 4 commits.
- ☐ Every message starts with a verb and says what changed.
- ☐ My git name is a nickname and my email ends in `@example.com`.
- ☐ There are no passwords or real names anywhere in my repository.